Integrating Handcrafted Features with Deep Representations for Smartphone Authentication

YUNPENG SONG, MOE KLINNS Lab, Xi'an Jiaotong University, China
ZHONGMIN CAI*, MOE KLINNS Lab, Xi'an Jiaotong University, China

Recent research demonstrates the potential of touch dynamics as a usable and privacy-preserving scheme for smartphone authentication. Most existing approaches rely on handcrafted features since deep models may be vulnerable to behavioral uncertainty due to the lack of consistent semantic information. Toward this end, we propose an approach to integrating handcrafted features into two phases of the deep learning process. On one hand, we present three fine-grained behavior representations by encoding semantic handcrafted features into the raw touch actions. On the other hand, we devise a deep Feature Regularization Net (FRN) architecture to combine the complementary information in both handcrafted and deep features. FRN involves handcrafted features as regularization to guide the learning process of deep features and selectively fuses these two feature types through a feature re-weighting mechanism. Experimental findings demonstrate that FRN outperforms the existing handcrafted or deep features even with smaller training and template sets. The framework also works for SOTA deep models and further boosts the accuracy. Results show that our approach is more reliable to alleviate behavioral variability and is competitively robust to statistical attacks compared with the most effective handcrafted features, suggesting a promising mechanism to improve the effectiveness and usability of behavioral authentication for multi-touch enabled mobile devices.

CCS Concepts: • **Security and privacy → Biometrics**; • **Human-centered computing → Ubiquitous and mobile computing**.

Additional Key Words and Phrases: Behavioral authentication, Feature fusion, Biometrics, Mobile devices

ACM Reference Format:
Yunpeng Song and Zhongmin Cai. 2022. Integrating Handcrafted Features with Deep Representations for Smartphone Authentication. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 6, 1, Article 27 (March 2022), 27 pages. https://doi.org/10.1145/3517332

1 INTRODUCTION

Smartphones have witnessed the rapid development of the mobile internet era, and leveraging smartphones to manage social life is currently developing an intuitive mindset for society. However, the increasing demand for easy access to smartphones poses challenges for the authentication system. According to Verizon Data Breach Investigation Report [55], in 2020, 67% of data breaches were caused by credential theft while 37% of breaches involved using stolen or weak credentials. With mobile devices carrying more sensitive information than ever, both public and private sectors are raising new security and privacy requirements and searching for appropriate authentication methods for mobile computing scenarios.

*Corresponding author.

Authors' addresses: Yunpeng Song, yunpengs@xjtu.edu.cn, MOE KLINNS Lab, Xi'an Jiaotong University, China; Zhongmin Cai, zmcai@sei.xjtu.edu.cn, MOE KLINNS Lab, Xi'an Jiaotong University, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2022 Association for Computing Machinery.
2474-9567/2022/3-ART27 $15.00
https://doi.org/10.1145/3517332

Knowledge-based passwords, such as Personal Identification Numbers (PIN) and pattern locks, are still the *de facto* standard method to verify users on a daily basis, e.g., during mobile payment [54]. However, with various devices and online services creating countless passwords, memorizing all those passwords puts too much workload on end-users and becomes a challenging task [44]. As a consequence, users tend to compose simple passwords or reuse passwords across multiple services as a workaround [51], leaving their passwords more vulnerable to attacks, such as brute-force attacks and dictionary attacks. Moreover, attacks from the insiders have become an important threat to be taken into account [55], yet the knowledge-based authentication is often susceptible to those attacks by leaving the smudge and chances of shoulder surfing the victim.

Another type of common authentication scheme is physiological biometrics, such as fingerprint, iris, and face recognition. This type utilizes physical characteristics to verify users, which proves to be accurate and relatively reliable during a long period. However, along with the high accuracy comes new privacy issues. Physiological traits such as fingerprints are persistent characteristics and basically can not be replaced, thus, an unexpected disclosure of physiological information may result in severe security hazards to the victims [63], especially in current society where faces or fingerprints are required everywhere without appropriate protections. Besides, the latest GANs show the capability of AI algorithms to break into the advanced physiological authentication systems with the success rates over 78% and 95% for fingerprint [5] and face recognition [26] respectively, alarming the security of physiological biometrics when facing attacks involving cutting edge AI technologies.

The above limitations motivate researchers to explore behavioral authentication methods for daily smartphone usage. Past work [12, 17, 39, 45] demonstrates that leveraging user's behavioral characteristics during interaction is a promising way to the convenient, privacy-preserving, and sufficiently secure authentication method. For example, touchscreen events and IMU data are commonly used for user monitoring. Frank et al. investigated the basic navigation touch operations when users interact with a smartphone for continuous authentication [17]. Shen et al. demonstrated the reliability and applicability of motion sensor data to build identity models for smartphone authentication [45]. Such behavioral authentication methods are perceived as usable and convenient with few privacy concerns, but sometimes the accuracy for verification is not high enough for practical applications [60].

The algorithms of behavioral authentication methods broadly fall into two categories, i.e., deep learning methods [53] v.s. handcrafted features with traditional machine learning [46]. In general, deep neural networks provide more powerful features for differentiating users and outperform traditional methods with handcrafted features. However, in some real scenarios, due to the changes in user's mental and physical status, their behaviors exhibit stochastic changes in the short run and unpredictable variability in the long run. Deep features lacking invariant semantic behavioral information are often not robust to these uncertainties. In contrast, handcrafted features which are designed to overcome the variations may contain consistent information across varied factors and thus lead to a more reliable result [50].

To improve the accuracy and security of the deep models, we propose a framework to integrate consistent semantic handcrafted features into two phases of a deep learning process. We hold the hypothesis that handcrafted and deep features are expected to have different error profiles, thus by regularization and selective fusion, more distinctive and consistent features can be learned from the above two feature types [8, 59]. However, most of the previous research on biometric information fusion employs the score level fusion method with manually fixed weights [1]. In the few research on feature level fusion, feature vectors of varied types are directly concatenated without adjusting the importance of different feature dimensions according to the dynamic behavior inputs [10, 62].

In our framework, we first present different behavior representation methods to characterize touch behaviors by encoding semantic handcrafted features. Then we propose a deep Feature Regularization Net (FRN) to automatically learn deep features complementary to handcrafted features and to assign varying weights according to the current input. By employing adaptive weights as scale factors, FRN selectively controls the impact of each feature dimension based on dynamic behavior inputs. After obtaining the regularized features from FRN, we employ distance-based KNN as the one-class classifier to make the final authentication decision. With a

dataset of 161 participants' touch behaviors over two months, we conduct extensive experiments to evaluate the performances of our proposal with respect to varied training set sizes, template numbers, behavioral variability, and attack scenarios. Our main contributions are highlighted as follows:

- We propose different behavior representation methods to characterize raw touch data. These methods explicitly integrate semantic features of touch data, such as physiological and kinematic features, into fine-grained behavior images or behavior sequences as input for deep networks. The behavior representations make it feasible to automate feature learning with deep algorithms for touch behaviors.
- We propose a deep Feature Regularization Net (FRN) architecture. FRN regularizes the learned deep features complementary to handcrafted ones and selectively combines the two feature types with a feature re-weighting mechanism. The final regularized features after fusion yield better authentication performances than each feature type when used individually.
- We examine the proposed method in terms of feasibility and effectiveness in a more practical scenario where only a small amount of legitimate samples and knowledge priors are available. With touch data from random users as knowledge priors, our approach significantly surpasses the accuracy of traditional machine learning methods. And leveraging the handcrafted features further reduces the knowledge priors required.
- We conduct extensive experiments to evaluate the proposed approach from the perspectives of sensitivity to training set size, usability to template number, impact of behavioral variability, and security under statistical attack. The results demonstrate the performances of our approach are reliable as a long-term authentication method in the scenario of both zero-effort attack and statistical attack.

The paper is organized as follows. Section II provides a brief review of related work. Section III introduces the touch data for authentication and handcrafted features extracted from touch behaviors. Section IV describes the behavior representations as inputs for deep algorithms. Section V demonstrates the details of the proposed FRN architecture. Section VI explains the training procedure of the model as well as training strategies. Section VII presents the experimental results and analyses. We briefly discuss the strengths and weaknesses in Section VIII. Finally comes the conclusion.

2 RELATED WORK

In this section, some prior works on behavioral biometrics are introduced. We divide the body of related work into two parts, i.e., handcrafted features with machine learning methods and deep learning algorithms. We summarize state-of-the-art studies that are most related to ours in Table 1.

2.1 Handcrafted Features with Machine Learning Methods

Owing to the popularity of touchscreens as a convenient input method, lots of work arises investigating touch-based authentication techniques [3, 7, 9, 12, 15, 39, 43, 47, 49, 50, 60, 65, 67]. These techniques explore the intrinsic differences in user's touch operations as features to verify users. For example, De Luca et al. [12] proposed a pattern lock with special constraints and analyzed the user's swiping sequences using XY-coordinates, pressure, size, time, and speed as features. Zhao et al. [67] proposed Graphic Touch Gesture Feature to represent movement and pressure differences among users. Cai et al. [7] extracted features including sliding speed, pressure variance, distance and angle between two fingers based on daily used two-finger gestures for quick authentication. Xu et al. [60] proposed and ranked lists of features across categories such as handwriting, pinch, and slide. Fierrez et al. [16] treated swipes as signatures and combined swipe feature set with signature feature set. Ooi et al. [37] processed the original feature vector with temporalization and randomization methods to retain the temporal touch information. Zhang et al. [66] integrated touch features (duration, distance, touch size, velocity) with

Table 1. Summary of related studies for touch behavioral authentication, with a breakdown of the studies into handcrafted feature-based methods and deep learning algorithms.

Category	Year	Study	Sensor	User #	Representation	Algorithm	EER(%)
Handcrafted	2013	Shahzad et al. [43]	to, acc	50	(sub-)stroke features	SVDE	4.8
	2017	Song et al. [50]	to	161	behav. + phyio. features	SVM/KNN	2.5-11.3
	2018	Fierrez et al. [16]	to	190	swipe + signature features	SVM+GMM	3.3-12.2
	2019	Ooi et al. [37]	to	190	stroke features	Temporal RF	4.0
	2020	Zhang et al. [66]	to, acc, gyro	41	sub-gesture features	MT-KNN	4.9
Deep	2019	Le et al. [28]	to	43	relative distance sequences	RNN	4.9-12.5
	2020	Tolosana et al. [53]	to	217	input time functions	TA-RNN	5.6-16.6
	2021	Shi et al. [48]	to, acc, gyro	77	polyline basic units	FCBBS	3.1-4.7
	2021	Lai et al. [27]	to	217	input time functions	Sig2Vec	4.3-9.1
	2021	This work	to	161	behavior images/sequences	FRN	1.5-4.1

Abbreviations have the following meaning: to = touch screen; acc = accelerometer; gyro = gyroscope.

motion features (rotation angle and moving speed in x-, y- and z-axis). They proposed a four-part based feature selection method with multi-threshold kNN to add another layer of protection over pattern lock.

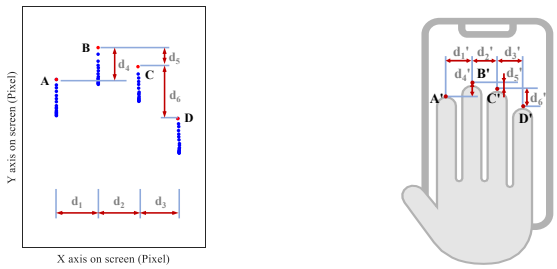
Features for user-defined multi-touch gestures are also investigated. Sae-Bae et al. [39, 40] first employed multi-touch gestures with all five fingers for large-sized tablet authentication. They designed and evaluated a set of 22 multi-touch gestures with X-Y coordinates and distances between fingers as features. Shahzad et al. [43] analyzed gestures with single or multiple fingers and proposed stroke and sub-stroke features including velocity, acceleration, duration, and displacement to verify users. They employed Support Vector Distribution Estimation as the one-class classifier. Sherman et al. [47] introduced free-form multi-touch gestures with any number of fingers drawing any shape for mobile authentication. They used principal component analysis and mutual information to verify users. Song et al. [50] defined a set of simple multi-touch gestures to estimate the hand geometry information. With these physiological features, their methods outperformed the commonly used touch behavioral features.

The prior research on touch behavioral authentication accumulates plenty of expert knowledge for feature engineering. These semantic handcrafted features may not yield performances as good as deep features in some cases, yet they might be more effective when facing behavioral variability or unmet (such as attack) scenarios [50].

2.2 Deep Learning Algorithms

Due to the powerful feature learning capability, deep learning algorithms applied in the domain of behavioral biometrics yield further performance improvements. For touch behavioral authentication, Le et al. [28] proposed using RNNs for the task of finger-drawn PIN code authentication. They transformed the raw touch coordinate data into stroke sequences of relative distance and feed the sequences into a bidirectional RNN. Similarly, Tolosana et al. [53] analyzed how users drew digital characters (PIN) on a touchscreen and proposed Time-Aligned Recurrent Neural Networks by combining Dynamic Time Warping with RNN. Acien et al. [2] treated user's free-text keystroke behaviors as sequences of four features (HL, RL, IL, PL) and applied a siamese RNN to verify users. Shi [48] built a set of polyline basic units for pattern lock gestures and divided the whole gesture into basic units. They developed a dedicated 1D-CNN based network to extract features and utilized a score-level weighted strategy for the final decision. Lai [27] also proposed a 1D-CNN model "Sig2Vec" for feature extraction, with dynamic time functions (like velocity and angle) of touch traces as input. Their approach learned the fixed-length representation from dynamic trajectories of arbitrary lengths and achieved state-of-the-art performance.

Other modalities for behavioral biometrics are also studied. For example, Chong et al. [11] investigated several deep learning architectures for mouse dynamics. They evaluate 1D-CNN, 2D-CNN, LSTM, and CNN-LSTM



(a) TFST physiological features (b) Real features of hand geometry

Fig. 1. An example of four-finger *TFST* gesture with physiological features and related real features of hand geometry.

on Balabit and TWOS datasets and recommended 2D-CNN although it seems counterintuitive. Xu et al. [61] leveraged active vibration signals of touching fingers to verify users from a physical perspective. They extracted CWT-based features in transient-state and cepstrum-based features in steady-state with a siamese network as classifier. Researchers also leveraged motion sensors to capture the different kinematics patterns of human movements for smartphone authentication. Neverova et al. [35] employed the inertial measurement unit in smartphones to record user's daily movements, and proposed an optimized shift-invariant Dense Convolutional Clockwork Recurrent Neural Network. Gadaleta et al. [18] presented a framework for gait recognition by building a CNN architecture for feature extraction and a one-class SVM for authentication.

Nevertheless, the prior work mainly relies on the raw sensor data sequences (XY coordinates of touch events and magnitudes of motion sensor events) or the stacked handcrafted feature sequences (movement speed, acceleration, continuous wavelet transform, etc.) as input of deep models. As a result, it is hard for deep models to learn globally consistent features with a strong generalization capability of handling intrinsic variability in user behaviors. Thus significant performance drops are observed when behaviors in the test set do not exhibit the same distribution as training data. To improve the effectiveness of the learned features and alleviate the impact of behavioral variability, we propose a deep Feature Regularization Net, which leverages invariant handcrafted features to regulate the deep feature learning process, to enhance the security and stability of the smartphone authentication.

3 TOUCH GESTURES AND HANDCRAFTED FEATURES

In this section, we will first introduce the touch behavior dataset used in this paper, and then describe the elaborate handcrafted features from both physiological and behavioral perspectives.

3.1 Dataset of Touch Gestures

We employ the multi-touch gesture dataset collected in [50] as our data source. The reasons are three folds: first, prior work has proven that multi-touch behaviors include rich identity information and thus the proposed framework can fully leverage the effective handcrafted features to regularize the learning of deep features; second, the dataset includes both behavioral and physiological features to describe the touch behaviors. These semantic features are well designed with good performances to serve as strong baselines; third, the dataset contains a long period of users' touch behaviors with a number of participants, which makes it feasible to evaluate the impact of behavior variability over the proposed method.

Specifically, touch gestures called *TFST* are used in this paper, which is short for "Touching with Fingers Straight and Together". Users perform gestures on the screen with two, three, or four adjacent fingers of a hand and gestures can be any shape, such as vertical lines and L-like shapes. We use the four-finger *TFST* dataset in this paper, which contains over 15k samples from 161 participants across six data sessions. A participant averagely contributes around 100 samples ($Median = 103$) with 17 samples per session after removing the erroneous ones.

3.2 Data Processing

Data processing is performed before the raw data is converted into behavior representations. Basically, two steps are included: touchpoint alignment and trace rotation. *1) alignment*: since the movements of varied fingers are slightly different, there exist situations in which one finger moves shorter/longer than the others. As the sampling frequency decreases for the certain finger when the touch screen detects no obvious movement, the number of touchpoints for each involved finger may not be the same, with outliers or missing points. To align the touchpoints, we sort them by timestamp and group the points with the same timestamp as a touch event. For incomplete touch events, we interpolate the events if only one point is missing (e.g., three touchpoints in a four-finger event) and remove the ones if more points are missing.

2) Rotation: *TFST* gestures place no restriction on input directions and users thus may perform the same gesture with various angles between the hand and the smartphone. To ensure comparability among samples of the same gesture, rotation is applied to the touch traces. We first compute the swiping velocity for each touch event along the traces to find the turning points ($velocity \leq 2$ *pixel/s*). Then a linear regression is employed on those points prior to the first turning points to estimate the initial directions of the strokes. A sample-wise rotation is exerted to make samples of the same gesture have a unified initial direction.

3.3 Handcrafted Features

The handcrafted features of *TFST* gesture consist of two categories, the physiological features representing the hand geometry information and the behavioral ones representing the dynamic swiping patterns. Both parts are explainable with semantic information. In this paper, we investigate how to merge handcrafted features with deep features and don't focus on searching for effective handcrafted features. Therefore, we select effective features proposed by previous studies and use them directly in our investigation.

3.3.1 Physiological Features. Take a four-finger *TFST* gesture as an example (Figure 1(a)). 12 distances are defined based on the traces as physiological features: AB, AC, AD, BC, BD, CD, d_1, d_2, d_3, d_4, d_5, and d_6. These features are shown effective to measure the real hand geometry in Figure 1(b) with high correlation coefficients [50]. For example, (A, B, C, D) is a set of touchpoints at the same timestamp in the traces, and when a user performs a four-finger *TFST* gesture naturally, the 12 physiological features are good estimates of corresponding real measurements of hand geometry, i.e., d_1, d_2, and d_3 in Figure 1(a) are close to the finger width d_1', d_2', and d_3' in Figure 1(b). And the 6 distances of AB, AC, AD, BC, BD, and CD in Figure 1(a) estimate the 6 fingertip distances of $A'B'$, $A'C'$, $A'D'$, $B'C'$, $B'D'$ and $C'D'$ in Figure 1(b).

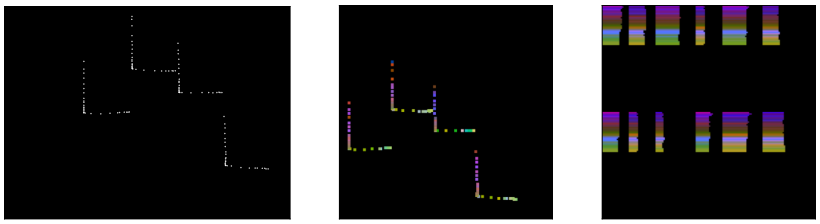
3.3.2 Behavioral Features. Prior studies (e.g., [38, 46, 64]) explore the feature space for touch dynamic and highlight the behavioral attributes of swiping length, velocity, timing, and pressure as generally effective features for authentication. In addition, angle-related features are also essential to describe the shape of a gesture trajectory. Therefore, we employ the above features and group them into the three categories: 1) trace shape-based (20-D): the angle- and length-related features to describe the shape of multi-touch traces; 2) motion kinematics (8-D): the velocity- and time-related features to present the kinematic information of swipe behaviors; 3) tactile pressure (24-D): the pressure-related features to show how hard the user touches the screen while performing the gestures, including the major axis of the touch area, the tool area, and the value of tactile pressure.

4 BEHAVIOR REPRESENTATIONS FOR DEEP NETWORKS

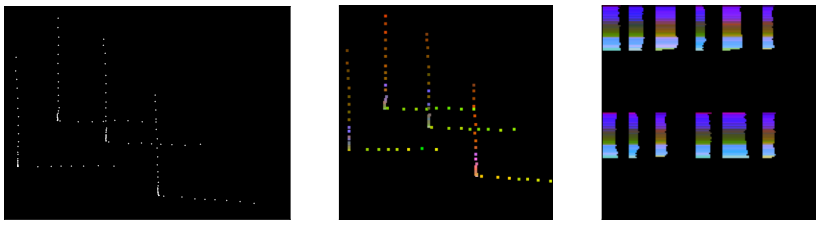
Well-designed deep architectures have demonstrated the strong capability of automatic feature learning, which yields even better performances than the handcrafted features by experts. However, since the user's touch sequences are usually not of the same size, it may not be a good way to directly feed the raw data into the deep networks as inputs. In this section, we propose different approaches to augment the raw touch data with semantic

Table 2. Brief Summary of Handcrafted Features

Categories	Features	Dimensions	Categories	Features	Dimensions
Physiological	Fingertip distance related	6	Behavioral	Trace shape based	20
	Finger width related	3		Motion kinematics	8
	Length difference related	3		Tactile pressure	24



(a) RTI example 1 (b) GTI example 1 (c) STI example 1



(d) RTI example 2 (e) GTI example 2 (f) STI example 2

Fig. 2. Example of (a/d) Raw Trajectory Images, (b/e) Graphic Touch Images, and (c/f) Statistical Touch Images. The first row display the images of a sample from participant 1 while the second row from participant 2. (b/e) and (c/f) are the inputs for deep networks. Black pixels are zero in value.

features and convert them into behavior representations as deep model input. The behavior representations consist of two categories, i.e., behavior images for CNN-based networks and behavior sequences for RNN-based networks.

4.1 Raw Trajectory Images

It is intuitive to use a snapshot of the raw touch trajectory to describe the touch behavior, which is referred to as Raw Trajectory Image (RTI) in this paper. The examples of Raw Trajectory Images are shown in Figure 2(a). When users perform a 4-finger *TFST* gesture, the screen will record the contact areas and report them in sequence as touch events. We denote the *n-th* touch event in the sequence as E_n and E_n contains multiple sub-events (i.e., E_n^0, E_n^1, E_n^2, E_n^3 for a 4-finger gesture). For the *n-th* sub-event in the sequence triggered by the *i-th* finger, we denote $E_n^i = (x_n^i, y_n^i, p_n^i, o_n^i, u_n^i, t_n^i)$, which stands for the XY-coordinates, tactile pressure, tool major, touch major of the finger i, and current timestamp, respectively.

E_n records each finger's position, pressure, tool major (related to the size of finger), and touch major (related to the size of finger contact area) according to the official Android API [13]. In a RTI, only the position information of touch events is marked as visual points on the zero-valued canvas background, as shown in Figure 2(a). In fact, the RTI does not contain behavioral traits like pressure and velocity patterns. Therefore, we propose three other behavior representations to convert the raw touch trajectories into augmented inputs for deep networks by encoding semantic features into representations.

4.2 Graphic Touch Image

Using the RTI as deep network input is simple but suffers from some limitations. For example, RTI does not contain some key features like pressure and the non-zero values are quite sparse in the whole image. This incomplete representation prevents the deep network to learn the most representative and effective features. Toward this end, we propose a two-step method to transform a RTI into a Graphic Touch Image (GTI).

4.2.1 Kinematic Information Encoding. The first step is to encode motion kinematics of touch behavior explicitly into the image. According to prior studies on touch features, we select three effective features (touch velocity, pressure, and duration) and encode their values into the three (RGB) channels of an image. Specifically, for a touch sub-event E_n^i triggered by the *i-th* finger, we divide the Euclidian distance between two consecutive touchpoints (x_n^i, y_n^i) and (x_{n+1}^i, y_{n+1}^i) by the time span Δt to get the velocity of the fingertip movement v_n^i. Then we normalize the value of v_n^i into a range between 0 and 255 to fit the first color channel intensity values R_n^i. Similarly, we encode the tactile pressure and duration information using p_n^i and t_n^i and normalize them to fit the second channel G_n^i and third channel B_n^i, respectively. To eliminate the influence of extreme values, the normalization method discretizes the values and linearly maps the middle 95% of the values into 255 levels. Values outside the range are mapped to either 0 or 255.

4.2.2 Spacial Information Encoding. The second step is to encode the spatial information in touch event sequences, such as shapes and distances, into an image. Since there are no restrictions on swiping areas when performing the gestures, the trajectories may start randomly in the top-left corner or in the middle, bringing inconsistency and confusion to the learning process of the deep model. To this end, we apply global translation to all touchpoints and move the whole trajectories to a fixed-size area. Take Figure 2(b) as an example, the trajectories are moved so that the leftmost turning points are always locating at the coordinates of (50, 350), aligning the trajectories across different images.

With modified coordinates, we build a zero-valued canvas sized 896×896 to highlight the colored trajectories. Points outside the rectangle are discarded. The size is selected to strike a balance between the data completeness and the sparseness, which covers all touchpoints of the majority of samples and eliminates zero-valued pixels as many as possible. To fit the input size of the ResNet-50, we linearly zoom out the image from 896×896 to 224×224. Also, the colored touchpoints are expanded from 1×1 to 3×3 pixels to make the data less sparse and more obvious to capture, as shown in Figure 2(b).

After the two-step method, a Raw Trajectory Image is augmented and transformed into a Graphic Touch Image (GTI). The process does not change the shape of the raw trajectories but highlights the kinematic and spatial information hidden behind the touch behaviors.

4.3 Statistical Touch Image

Graphic Touch Images focus on describing the kinematic and spatial information in touch behaviors but ignore the hand geometry traits hidden in the gestures. Thus, the Statistical Touch Image (STI) is proposed with image textures and silhouettes explicitly revealing the semantic physiological hand geometry traits and behavioral characteristics (Section 3.3.1).

4.3.1 Physiological Information Encoding. In STI, the aforementioned 12 physiological features (Section 3.3) are embodied with visible lines of varied lengths calculated from the trajectories. Take the feature AB in 1(a) as an example. We first measure the distance d_n^{AB} between (x_n^0, y_n^0) and (x_n^1, y_n^1) by Euclidean distance. Then we normalize d_n^{AB} into the range of $[1, 30]$ with a linear mapping function and get the length value l_n^{AB}. In this way, feature AB is encoded into a visible line sized $1 \times l_n^{AB}$ in the image. Therefore, a touch event E_i with four touchpoints is visualized with 12 lines whose lengths are proportional to the corresponding hand geometry measurements. To fit the input size of deep models, the first six lines are positioned in sequence at the *i-th* row

on the upper part of the image while the latter six lines are at *(120+i)-th row* on the lower part. As each line is 1 pixel in height, the total height of the line block depicts the number of touch events in a gesture, which also reflects the time duration.

4.3.2 Behavioral Information Encoding. Besides physiological features, behavioral attributes such as swiping patterns and rhythms are also displayed in STI. Similar to GTI, we paint the 12 lines with RGB colors whose intensity values are used to encode the velocity of the touching fingertip, tactile pressure, and duration. The above features are mapped to 256 levels to meet the requirement of the visible color intensity. For each block of lines, the changing tendency of the visible color shows the outline of the user's fingertip velocity as well as tactile pressure and duration during swiping the gesture. For example, the more red components in the image mean a faster swipe while the more blue components mean a harder press.

Through encoding the above two types of information, all the touch events of a four-finger *TFST* gesture make up 12 separate blocks in the image, with silhouette representing the physiological traits and colors showing the behavioral characteristics. The STI is not as easy as the RTI for a human to understand, but it highlights the statistical details in both physiological and behavioral aspects.

4.4 Touch Dynamic Sequences

Apart from transforming the sequences into behavior images, we also propose Touch Dynamic Sequences (TDS) as the behavior representation to leverage the temporal dependency in time sequences. Similar to STI, we highlight both physiological and behavioral traits to augment the time sequences. Take a four-finger *TFST* gesture as an example, we denote the *n-th* touch event in the sequences as E_n. For the physiological part, the 12-dimension physiological feature vector (Section 3.3.1) of E_n is computed using the XY-coordinates of four sub-event E_n^0, E_n^1, E_n^2, and E_n^3. For the behavioral part, dynamic features, such as velocity, pressure, and duration of the touch behaviors, are computed for each sub-event. Note that the duration values of four sub-events are almost the same, so we use the average instead of four separate values. Therefore, a 21-dimension feature vector is extracted to align with the touch event E_n, including 12 dimensions for physiological traits encoding and 9 for behavioral ones. The length of the TDS is set to 120 to keep the majority of the samples completed. Samples less than 120 events are padded with zeros while those more than 120 events are truncated. So the TDS is formulated by a series of embeddings sized 120×21 as inputs to BiLSTM models.

5 MODEL ARCHITECTURE

In this section, we will introduce our proposed Feature Regularization Net (FRN) which selectively combines deep features with handcrafted ones to get the final behavior descriptor. The model architecture is shown in Figure 3. FRN includes two separate feature extraction parts to get handcrafted and deep features, followed by a regularization and fusion part for deep feature reconstruction and selective fusion with handcrafted ones. FRN outputs the regularized behavior descriptor for the subsequent training and inference stages.

5.1 Feature Extraction

The feature extraction part aims to generate discriminative feature representations for the touch behaviors. The Feature Regularization Net includes two types of feature extraction techniques, i.e., the handcrafted feature engineering technique and the DNN-based automatic deep feature learning technique. For deep feature extraction, ResNet-50 is employed for behavior images (GTI, STI) and BiLSTM for behavior sequences (TDS).

5.1.1 Handcrafted Feature Extraction. As discussed in 3.3, for a four-finger *TFST* gesture, we extract a 64-dimension handcraft feature vector including both physiological and behavioral characteristics. While the physiological features lay the foundations for verifying users, its behavioral counterparts are able to supplement

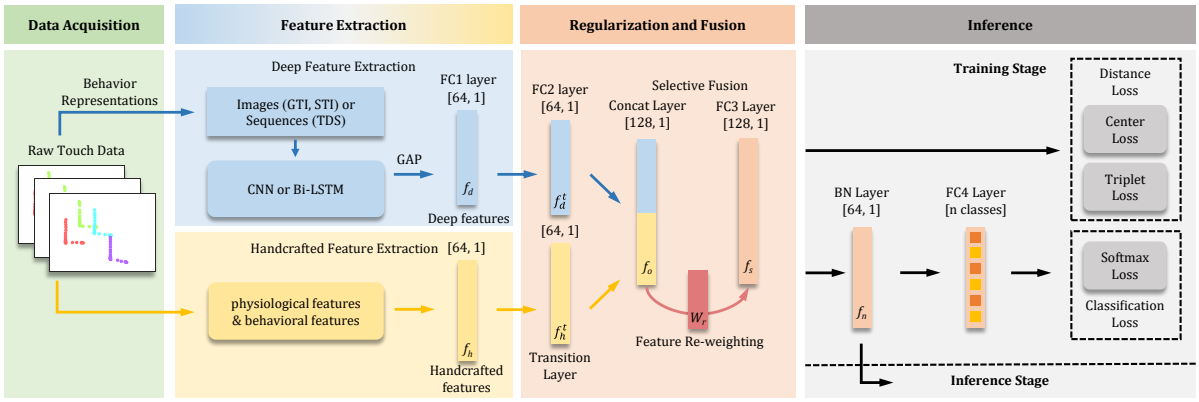


Fig. 3. The architecture of Feature Regularization Net (FRN). The semantic handcrafted features influence the network through the deep feature extraction phase (blue) and the regularization and fusion phase (orange).

the identity information from a dynamic perspective. Therefore, both of them are employed as handcrafted features (f_h in the architecture in Figure 3) to regularize and complement the deep features.

5.1.2 Deep Feature Extraction with CNN. In 4.2 and 4.3, we convert touch data into behavior images to satisfy the input requirement of CNN models. Note that most of the touch behavior datasets are relatively small in size, to avoid the overfitting problem, it may not be the best practice to train a large model from scratch. To address the issue of limited samples, we utilize a pre-trained model and finetune its parameters to fit the data distribution of our inputs. Specifically, we build the ResNet-50 [20] via PyTorch 1.2.0 as CNN feature extractor and load the weights[1] pre-trained by ImageNet for initialization.

ResNet-50 network is originally designed for the multi-class classification task, which tries to assign a correct label to the image from a set of labels. However, in the authentication task, the algorithm only needs to verify whether the current user is the genuine one or the imposter, thus we modify the ResNet-50 network into a feature extractor. We remove the softmax layer and replace the final fully-connected layer with a new one of 64 hidden nodes. In this way, the ResNet-50 network is no longer a classifier in our method, instead, it serves as a feature extractor and outputs 64-dimension deep features. The dimension of deep features is consistent with the handcrafted ones to balance the impact of both feature types. We use ADAM optimizer [25] to optimize losses and update the parameters. The learning rate is initialized as 6×10^{-3} and decreases every 30 epochs by a factor of 0.1 before reaching the maximum 50 epochs. The mini-batch size is tested from 16 to 512 and finally set to 128 to cover the efficiency and generalization. We also utilize the CUDA toolkit (CUDA 10.0.130 and cuDNN 7.6.2) with 8 Tesla P40 GPUs to accelerate the computation process.

5.1.3 Deep Feature Extraction with BiLSTM. Since the proposed TDS (4.4) consist of a sequence of touch events with temporal dependency, we employ Bidirectional Long Short-Term Memory (BiLSTM) [19] as another deep feature extractor for the automatic feature learning. LSTM effectively overcomes the vanishing gradient problem from which vanilla RNN suffers by introducing dynamic gates to save or forget information. Past work reveals that LSTM models with two layers slightly outperform the others [23], and the capability of preserving bidirectional information from both left and right sides assists the network to generate more representative features [19]. Given the above empirical observations, in this work, we employ a two-layer BiLSTM network.

For presenting a touch gesture, it is critical to capture the dependencies between steps in TDS since the relative positions of touchpoints at each time step are basically determined by the hand geometry. Therefore, the

[1]https://download.pytorch.org/models/resnet50-19c8e357.pth

BiLSTM network is employed to build a robust mapping between hand geometry and deep feature space due to its capability of learning bidirectional dependencies. Similar to the CNN model, the BiLSTM is implemented in PyTorch 1.2.0 with 8 GPUs to speed up. The input size is fixed to 120, i.e., sequences shorter than 120 are padded with zeros as described in 4.4 while longer sequences are truncated. The size of the hidden states at each time step is 32 for each direction to generate a 64-dimension deep feature vector whose size is comparable with handcrafted features. Dropout layers ($p = 0.4$) are added between the LSTM layers to prevent overfitting. We apply the Tanh function as the activation function. The settings for optimizer and mini-batch size are exactly the same as those used for the CNN network while the base learning rate is initialized as 1×10^{-3} and decays by a factor of 0.1 every 30 epochs before reaching 50 epochs.

5.2 Regularization and Fusion

As shown in the left part of Figure 3, each of the two feature extraction techniques outputs a 64-dimension feature vector, the automatically learned deep feature vector f_d and the handcrafted feature vector f_h, respectively. We introduce feature regularization part to supervise the deep features learning process and force f_d complementary to f_h. In the regularization process, a fusion part is also introduced to selectively combine handcrafted and deep features conditioned on specific samples.

5.2.1 Feature Transition and Selective Fusion. Since the handcrafted features f_h and deep features f_d are extracted with heterogeneous methods, the features f_h and f_d display notable discrepancy in distribution. Therefore, we add a fully-connected layer as the transition layer following the features. The transition layer serves as a filter to map f_h and f_d into a more unified space. The size of the transition layer is 64 to match the size of the original f_d and f_h. With f_d^t and f_h^t denoting the outputs of the transition layer, we concatenate the two transitioned feature vectors as f_o,

$$f_o = [f_d^t; f_h^t] \tag{1}$$

Rather than using manually set weights to combine the f_d^t and f_h^t, we propose a selective fusion approach that re-weights the features dynamically based on current inputs. A re-weighting matrix (or vector) W_r is jointly trained in an end-to-end manner with f_o involving. Depending on the re-weighting strategies, the concatenated feature vector f_o is multiplied or element-wise multiplied with W_r to selectively control the impact of each feature dimension in the pipeline,

$$f_s = \sigma(W_r * f_o) \tag{2}$$

where σ refers to the ReLU activation function; the production of W_r and f_o applies the matmul product (matrix product) for re-weighting matrix or the hadamard product (element-wise product) for re-weighting vector. Parameters in the network are updated with ADAM optimizer automatically. Without loss of generality, the remaining parts are discussed by the example of re-weighting matrix.

5.2.2 Regularization of the Handcrafted Features. We will briefly demonstrate how the handcrafted features f_h regularize the learning of deep features f_d to make it more complementary from the perspective of the parameters updating, i.e., network parameters are computed with the impact of f_h. We use J to denote the loss of the task and W_{ij}^l to denote the weight associated with the connection between node j in layer l and node i in layer $l+1$. z_i^l denotes the total weighted sum of inputs to node i in layer l, and a_i^l refers to the activation of node i in layer l with $h(\cdot)$ as the activation function. Then we can get $z_i^{l+1} = \sum_j W_{ij}^l a_j$, and $a_i^l = h(z_i^l)$. We define an error term δ_i^l that measures the extent to which node i in layer l contributes to the errors in the loss. For each node i in layer l,

$$\delta_i^l = (\sum\nolimits_j W_{ji}^l \delta_j^{l+1}) h^{'}(z_i^l) \tag{3}$$

and the partial derivatives is computed as

$$\frac{\partial J}{\partial W_{ij}^{l}} = a_j^l \delta_i^{l+1} \tag{4}$$

In our cases, for the FC1 layer where $a = f_d$ and $l = 1$, we have

$$\frac{\partial J}{\partial W_{ij}^{1}} = f_{d(j)} \delta_i^2 \tag{5}$$

where

$$\delta_i^2 = (\sum\nolimits_j W_{ji}^2 \delta_j^3) h^{'}(z_i^2) \tag{6}$$

$$\delta_i^3 = (\sum\nolimits_j W_{ji}^3 \delta_j^4) h^{'}(z_i^3) \tag{7}$$

as we can see, the partial derivatives of the loss function is affected by the handcrafted feature f_h as follows,

$$z_i^2 = \sum\nolimits_j W_{ij}^1 f_{d(j)} + \sum\nolimits_j \hat{W}_{ij}^1 f_{h(j)} \tag{8}$$

$$z_i^3 = \sum\nolimits_j W_{ij}^2 h(\sum\nolimits_k W_{jk}^1 f_{d(k)}) + \sum\nolimits_j \hat{W}_{ij}^2 h(\sum\nolimits_k \hat{W}_{jk}^1 f_{h(k)}) \tag{9}$$

Given the above analysis, the handcrafted features f_h affect the learning process of deep features f_d in both forward and back propagation processes. For example, the loss J is computed directly with f_h in the forward propagation process; by back-propagating the loss through z_i^2 and z_i^3, the algorithm also tracks f_h to update the network parameters.

5.3 Loss Function

In real scenarios, samples for inference usually have discrepancies with the training set due to the unpredictable human behavior variability, degrading the authentication accuracy of behavioral authentication methods. A common practice is to enlarge the margin between classes to mitigate the discrepancy issue [56]. Therefore, we propose a joint distance and classification loss to optimize the learned features. The distance loss is made up of a triplet loss and a center loss while the classification loss utilizes the softmax loss. The triplet loss tends to enlarge the relative distance between samples of different classes and the center loss encourages reducing the distances within the same class. However, they are more likely to be impacted by local samples [33]. On the other hand, the softmax loss globally sharpens the edges between different classes. Given the above considerations, we jointly train the two types of losses aiming at leveraging their complementary advantages to avoid local misalignment.

The triplet loss inputs three samples together for measurement to force the samples from the same class to be closer while pushing samples from the different classes away. It focuses on the relative distance and improves the robustness of the identity information in features. Triplet loss is formulated as:

$$\mathcal{L}_t(a, p, n) = [D_{ap} - D_{an} + \alpha]_+ \tag{10}$$

where a, p, n denotes the anchor, positive, and negative sample, respectively. D_{ap} and D_{an} are the distance measurements of positive pair and negative pair, respectively. α refers to the margin and $[x]_+$ stands for the function of $max(x, 0)$. Triplet loss optimizes the relative distance measurements between random positive pair and negative pair yet ignores the absolute values.

Center loss [58] reduces the variance of learned features within the same class by assigning a center for each class and penalizing the distances between features and corresponding centers. The center loss is formulated as:

$$\mathcal{L}_c = \frac{1}{2} \sum\nolimits_{i=1}^{B} \left\| x_i - c_{yi} \right\|_2^2 \tag{11}$$

where B is the mini-batch size and c_{yi} denotes the y_i-th class center of deep features. The formulation above effectively characterizes the intra-class variations and the center c_{yi} is computed by averaging the features of the corresponding classes in a mini-batch.

Since center loss ignores the inter-class distances, we employ the softmax loss as the classification loss. Softmax loss leverages the global characteristics of different features to ensure the margins between classes. Also, it is shown to be beneficial for loss convergence and performance improvement. Softmax loss is given as:

$$\mathcal{L}_s = -\sum_{i=1}^{B} log\frac{e^{W_{y_i}^T x_i}}{\sum_{j=1}^{n} e^{W_j^T x_i}} \tag{12}$$

where W_j denotes the prediction function parameter of the training class k.

The joint loss $\mathcal{L}$ consists of softmax loss, triplet loss, and center loss as follows:

$$\mathcal{L} = \mathcal{L}_s + \lambda_1 \mathcal{L}_t + \lambda_2 \mathcal{L}_c \tag{13}$$

where scalar λ_1 and λ_2 are weights for controling the impact of different losses. We briefly demonstrate the supervised learning process with the proposed loss in Algorithm 1.

Algorithm 1 The brief learning process

Input: Training data x_i. Parameters θ of the feature extractor, W, c_j in the loss layers, λ_1, λ_2, α, learning rate γ.

Output: Parameter θ

while not converge **do**
  $n \leftarrow n + 1$
  Compute the loss: $\mathcal{L}_s^n, \mathcal{L}_t^n, \mathcal{L}_c^n, \mathcal{L}^n$; $\mathcal{L}^n = \mathcal{L}_s^n + \mathcal{L}_t^n + \mathcal{L}_c^n$
  Compute the error: $\frac{\partial \mathcal{L}^n}{\partial x_i^n} = \frac{\partial \mathcal{L}_s^n}{\partial x_i^n} + \lambda_1 \cdot \frac{\partial \mathcal{L}_t^n}{\partial x_i^n} + \lambda_2 \cdot \frac{\partial \mathcal{L}_c^n}{\partial x_i^n}$
  Update W: $W^{n+1} = W^n - \gamma \cdot \frac{\partial \mathcal{L}^n}{\partial W_i^n} = W^n - \gamma \cdot \frac{\partial \mathcal{L}_s^n}{\partial W_i^n}$
  Update c_j for each class j: $c_j^{n+1} = c_j^n - \alpha \cdot \Delta c_j^n$
  Update θ: $\theta^{n+1} = \theta^n - \gamma \sum_i^m \frac{\partial \mathcal{L}_t^n}{\partial x_i^n} \cdot \frac{\partial x_i^n}{\partial \theta^n}$
end while

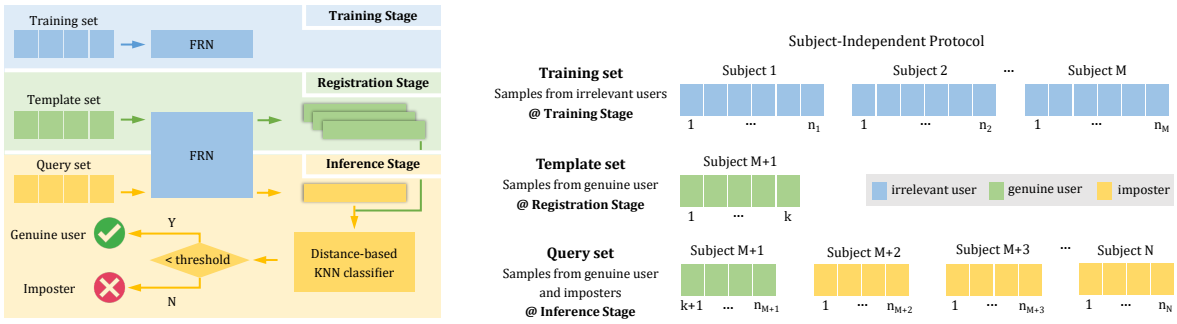
6 AUTHENTICATION PROCEDURE AND TRAINING STRATEGIES

In this section, we will describe the authentication procedure in our architecture and introduce the training strategies employed for mining samples and preventing overfitting.

6.1 Authentication Procedure

Since getting data from potential attackers is a hard task in the wild, it is more practical to build the model only with data from the genuine user and a few knowledge priors. Therefore, in this paper, we prepare a common training set with samples from irrelevant users as knowledge priors and train the FRN only with those data. After a genuine user first registers with touch gestures as template set, the follow-up authentication attempts will be sent into the FRN and the output will be compared with those in the template set to infer the identity. According to the above process, we divide the authentication procedure into three stages, i.e., training stage, registration stage, and inference stage, as Figure 4 shows.

The training stage teaches the FRN to discover the intrinsic differences in identities and further to learn holistic features from a crowd of users. Note that the training stage will only appear once due to the practical requirement, so we first establish a common training set at this stage. The training set includes touch data from



(a) Authentication procedure of the proposed approach. The training stage is only required once.

(b) The subject-independent protocol for experiment evaluation. The genuine user and imposters do not appear in the training set.

Fig. 4. An overview of dataset protocol design and authentication procedure.

M random users who are irrelevant to the genuine user. Once the network is trained, it maps touch data into feature descriptors representing the identity. At the registration stage, the genuine user is asked to perform *TFST* gestures multiple times to establish the legitimate template set. The templates will be fed into the FRN to generate feature descriptors as the genuine profile. Ideally, the template set should cover a diversity of samples to track the behavioral patterns of the genuine user. At the inference stage, we refer to an authentication attempt as a query. The query will be sent into the network and computed feature distance with descriptors in the genuine profile through a distance-based K Nearest Neighbor algorithm. If the feature distance is less than the threshold, the query will be inferred as the genuine user, otherwise the query will be rejected as an imposter, the whole procedure shown in 4(a).

6.2 Subject-independent Dataset Protocol

As the dataset consists of touch data from 161 participants, in the following evaluation, we build the subject-independent data protocol as Figure 4(a) shows. We first randomly take out M users and utilize their samples to establish the training set. Then we designate one of the remaining participants as the genuine user while the others ($161 - M - 1$ participants) as impostors. It is noted that samples of the genuine user are not included in the training set to fairly evaluate the generalization capability of our method. The template set only contains a proportion of positive samples from the genuine user while the query set includes the remaining positive samples and thousands of negative samples from more than 100 imposters (if $M \leq 60$). The procedure is repeated until each of the $161 - M$ participants outside the training set is designated as the genuine user in turn. The whole procedure is repeated 10 times with independently selected M users for the training set to account for randomness.

Note that in Section 7, handcrafted (physiological and behavioral) features which are proved effective serve as baselines for proposal evaluation. Different from the procedure of FRN, the pipeline of these baseline methods only consists of two parts, handcrafted feature extraction for representation and distance-based K Nearest Neighbor for verification. The procedure excludes the training set built from random users in that no general model is required. Therefore, the implementation of the baseline methods only includes the template set and the query set, which follow the same protocol as the FRN shown in 4(a).

6.3 Batch Sampling Strategy

Compared to softmax loss, triplet loss in the joint loss function is much harder to train [57]. Meanwhile, the behavioral variability along time is a critical factor leading to high intra-class variance and deteriorating the

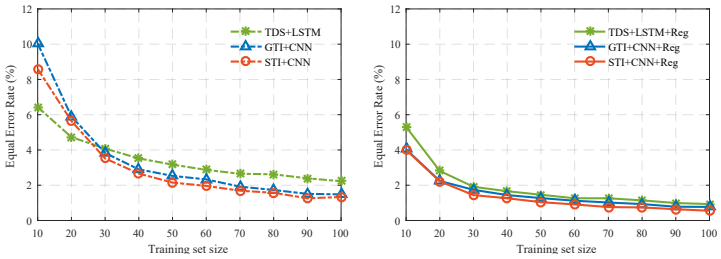


Fig. 5. EER curves for different deep features learned at varying training set sizes. Left: deep features without regularization; right: regularized deep features.

long-term authentication performance. Given the above two concerns, we first employ the mini-batch [6] for gradient computation to accelerate loss propagating throughout the network and speed up the convergence process. Then we design the hard positive mining strategy leveraging the temporal information to search for a diversity of positive samples that cover the user's behavior patterns in a more accurate manner.

The size of the mini-batch is set to B samples from random P users in the training set with K samples per user ($B = P \times K$). As for mini-batch sampling, it is a common practice to utilize hard negative samples for training, however, in our work, we focus on mining hard positive samples in different sessions, i.e., K samples of a user are required to derive from at least two different sessions. The reasons we set sessions as the criterion rather than feature distances are two folds. First, triplets with the largest feature distance $\left\|f(x) - f(x_p)\right\|_2^2 - \|f(x) - f(x_n)\|_2^2$ are quite sensitive to the local sampled data and fitting them perfectly may lead to a global performance decrease. Second, time is the key descriptor to behavioral variability. Samples from different sessions are more representative to describe the varied behavior patterns of the same user along time, guiding the model to learn an accurate representation from a time-independent perspective. Based on [32], a large value of K is more likely to find hard positives while large P is beneficial to search for hard negatives. So K and P are set to 16 and 8 to balance our sample mining goals.

7 EXPERIMENTS AND EVALUATION

In this section, we conduct extensive experiments to evaluate the reliability and security of our approach. We also systematically investigate the factors that affect the performances with an ablation study and compare FRN with the state-of-the-art methods.

7.1 Impact of Training Set Size for Deep Feature Extraction

Unlike the handcrafted features which are selected manually and directly fed into the KNN classifier at the inference stage, the deep feature extractor in our framework requires training to learn the effective representations in the form of deep features at the training stage. In this section, we investigate the training size effect in deep model feature extraction, aiming to examine whether it is feasible to build a general model for different users with a small training set.

To answer this question, we experiment with the number of users in the training set as the independent variable, exploring the impact of training set size on authentication performances. More precisely, the training set size fits into [10, 100] irrelevant users with a step of 10. Each irrelevant user contributes about 100 touch samples to the training set while the template set consists of 30 touch samples from the genuine user. The inputs of the model include behavior images (GTI and STI) and behavior sequences (TDS), each of which is processed with and without regularization. The experiment is repeated 10 times to account for the randomness. Figure 5 depicts how the authentication performances change with the training set size.

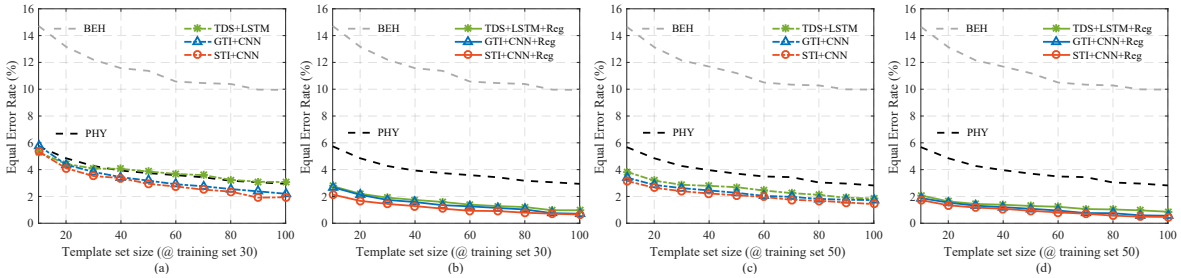


Fig. 6. EER curves for different features at varying template set size. Handcrafted behavioral (BEH) and physiological (PHY) features serve as baselines. Training set size is 30 for (a/b) and 50 for (c/d); (a/c) uses deep features alone without regularization while (b/d) uses regularized deep features.

Unsurprisingly, we can easily see the trend that larger training set teach better models. Deep feature extractor learns from samples of irrelevant users by translating these shared knowledge priors into suitable network parameters that yield more representative features. The observation also demonstrates the feasibility of building a general model for authentication. With data from irrelevant users as shared knowledge priors, genuine users only need to enroll their *TFST* gestures several times and the general model could verify their identities at relatively high accuracy. Notably, by regularizing the deep features with handcrafted ones and selectively fusing them, FRN combines the complementary information of both feature types, so the regularized deep features achieve lower EERs than the deep features without regularization. Moreover, the involvement of handcrafted features in the architecture reduces the size of the training set demanded by a large step. For example, FRN with regularization can guarantee an EER of 2% for a training set of 30 users while achieving the same EER level without regularization demands a training set of more than 80 users. FRN provides an effective way to decrease the irrelevant user number, making the deployment in the wild more practical.

7.2 Impact of Template Set Size

In this section, we will investigate the impact of template set size on the authentication performances. We use a fixed training set to build the model and vary the template set size as the independent variable. The template set size fits into $[10, 100]$ samples with a step of 10. Similarly, the inputs of the deep model include GTI, STI, and TDS, each processed with and without regularization. Handcrafted behavioral and physiological features serve as two baselines in contrast to the learned deep features. The experiment is repeated 10 times to mitigate the influence of randomness. And the configuration for training deep models stays the same as introduced above. Figure 6 illustrates the average EER performances of different features with respect to varied template set sizes.

In the figure, we can observe a strong and clear performance tendency, i.e., with more templates characterizing the genuine user's behaviors, the models are more accurate with lower EERs. For deep features, a sharp decrease in EER at the beginning of the curves is also observed, making it feasible to ensure fair security with only a few templates enrolled at the registration stage. For instance, when a new user registers, she could enroll 20 templates to achieve EERs around 2% using regularized deep features or 4% using deep features without regularization. With more available templates after registration, the accuracy could be further improved to a much higher level.

Although the physiological features demonstrate equivalent performances as the deep features without regularization when the training set size is 30 (Figure 6(a)), it is easy to find that the deep features outperform the physiological ones by a large margin when the training set size grows to 50 (Figure 6(c)). More importantly, handcrafted feature regularization with FRN makes significant progress in decreasing EERs compared with deep features alone. The regularized features need much fewer templates to reach a similar level of accuracy as the other feature types do. For example, achieving 2% of EER demands 10-20 templates for regularized features, or 100 templates for features without regularization, or far more templates for handcrafted ones. We attribute this to

Table 3. Settings of the Template Set and Query Set for Session N

Datasets	Description
Template set	5 samples of the participant in Session 1
Query set	Genuine: samples of the participant in Session N Illegitimate: samples of all imposters from all sessions

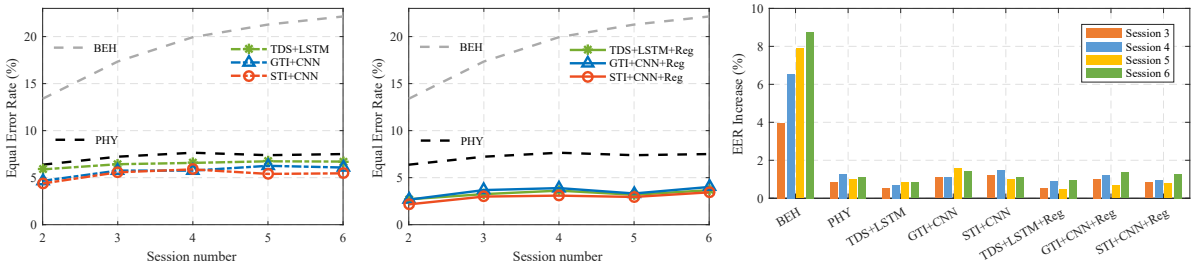


Fig. 7. Long-term EER curves for features. Handcrafted behavioral (BEH) and physiological (PHY) features serve as baselines. Left: deep features without regularization; middle: regularized deep features; right: average EER increases in Session 3-6 compared with EERs in Session 2.

the fact that the handcrafted and deep features are expected to have different error boundaries, thus regularizing and selectively fusing with them yield higher accuracy than either one alone.

7.3 Impact of Behavioral Variability

Long-term behavioral variability is one of the most critical challenges hindering the adoption of behavioral biometric techniques in real-world scenarios [60]. In this experiment, we will investigate whether the behavior representation methods and FRN facilitate deep features to resist behavioral variability over time.

As introduced in Section 3.1, the data collection procedure lasts for two months with six data sessions. The period between two consecutive sessions is more than one week to guarantee that substantial amounts of behavioral variability in long-term behaviors are captured. With the multiple-session touch dataset, we can establish the template set and query set with data from different sessions to simulate the long-term usage cases. The settings of template set and query set for Session $N \in \{2, 3, 4, 5, 6\}$ are shown in Table 3. For each genuine user, 5 samples in Session 1 are selected to build the template set. Five query sets (2-6) are built for evaluation. Each query set includes the genuine samples in the corresponding session (2-6) and all the imposters' samples regardless of sessions.

Similar to the previous experiments, the training set comprises 30 irrelevant users to balance the tradeoff between usability and security. We test three types of behavior representations (GTI, STI, and TDS) in this experiment, all of which are processed both with and without regularization. The handcrafted behavioral and physiological features are also employed under the same condition as baselines to illustrate the effectiveness of the deep features. The experiment is repeated 10 times to account for the randomness. The settings for the deep networks stay the same as introduced above. Figure 7 displays the performances in average EERs with respect to queries from different sessions and the EER increases regarding sessions.

The figure clearly displays that the EERs for behavioral features have a significant growth as the time elapses while the performances of physiological features stay quite consistent during the same periods, only with a slight EER increase. This basic observation indicates that behavioral variability severely influences the authentication accuracy of behavioral features from a long-term perspective. However, it also highlights the opportunity to

address this challenge: leveraging invariant information such as hand geometry features may largely alleviate this type of uncertainty over time.

It is interesting to note that deep features without regularization perform slightly better than physiological features. Also, the EER increases (EERs in Session 2 subtract from EERs in Session 3-6) shows that long-term intervals between sessions do not bring large EER rises for the deep features, differing from the EER increase trend of behavior features which keeps a significant growth along time. This result proves again the effectiveness of the proposed behavior representations, which explicitly encodes consistent features that are not easily changed over time. Moreover, the three types of regularized deep features yield the best performances among all sessions. It indicates that FRN is able to fully extract the time-irrelevant consistent information in handcrafted features and improves the quality of deep features by enhancing the resistance against unpredictable behavioral variability.

7.4 Performances under Statistical Attack

The above performances are evaluated under the zero-effort attack scenarios when the attackers do not elaborately imitate the genuine user's behavior pattern. Besides the zero-effort attack, it is also important to consider attack cases where malicious users have knowledge of the authentication method and aim to attack the system with deliberate attempts. There are several types of knowledgeable attacks threatening the behavioral authentication, i.e., shoulder surfing attack [24], smudge attack [4], and statistical attack [42], of which the statistical attack is considered to be severest to the behavioral methods [42]. Previous work also shows the resistance of multi-touch gestures against the shoulder surfing and smudge attack [40, 50]. Given the above considerations, in this paper, we evaluate our method in the scenario of the statistical attack.

The intuition of statistical attack comes from the observation that the feature values of a user's multiple touch attempts fall into a certain range. If the range is large, the feature values of the user may have a high possibility to overlap with the average values across a crowd of people. This observation suggests that leveraging the statistics of the population could generate advanced forgeries to fool the algorithms with a higher success rate. For example, Serwadda et al. demonstrated how to use a single Lego robot to increase the EERs of a touch-based authentication system by between 339% and 1,004% [42]. With this intuition, we generate statistics-based synthesis based on the data from a crowd and conduct the statistical attack for security evaluation.

Algorithm 2 Creating forgeries for statistical attack

Input: Feature matrix of a population of users *RealFea*, number of bins for each feature dimension *NumOfBins*.
Output: Forged feature matrix *ForgedFea* for attack.

while n < *NumOfFea* **do**
    $n \leftarrow n + 1$
    BinBnd, *BinFreq* = Binning(*RealFea*[n], *NumOfBins*);
    # generate bins according to *RealFea* and *NumOfBins*
    HighFreqBin = SortBinsByFrequency(*BinBnd*, *BinFreq*);
    # find the bin of the highest frequency
    LowerBnd,*UpperBnd* = GetBound(*HighFreqBin*);
    # get the bound of the bin
    ForgedFea[n] = uniform(*LowerBnd*,*UpperBnd*);
    # Generate random values for *n-th* dimension of feature.
end while
Return *ForgedFea*

To generate forgeries of high quality, in this work, we skip the process of using the programmed robot to physically swipe on the touchscreen, instead, we directly generate attack feature vectors to feed the distance-based

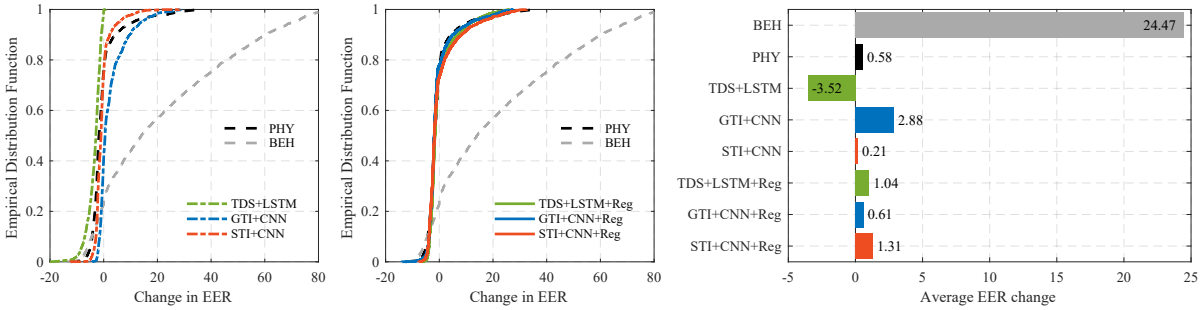


Fig. 8. Impact of statistical attacks in contrast to zero-effort attack. For each legitimate user, we subtract the EER under zero-effort attack from that under statistical attack as the EER change. Left: deep features without regularization; middle: regularized deep features; right: average EER changes for different features.

KNN classifier by assuming a perfect robot. More specifically, we first estimate the empirical probability density function of features from a crowd of users. Then we sample the feature values which are of high possibility to appear in most users' behaviors. With these feature values, we create forgeries to conduct the statistical attack, as shown in Algorithm 2. The inputs of Algorithm 2 include a matrix of the real features *RealFea* and a hyperparameter *NumOfBins*. *RealFea* consists of feature vectors extracted from a large population. Each column is the feature vector of a real sample while each row represents a series of values for one feature dimension. For *RealFea*[n], the *n-th* row of *RealFea*, the values are binned to approximate the empirical probability density function. *NumOfBins* is the variable controlling the granularity of approximation. The bin with the highest frequency is used for sampling and generating forged feature values for the statistical attack.

We utilize a training set of 30 participants for model training and generate forged feature vectors for attack evaluation from a population of all 161 participants. The *NumOfBins* is set to 20 after applying a grid search for the optimal variable to guarantee the quality of the statistical attack. For each of the 131 users out of the training set, we randomly selected 30 samples as the genuine user's template set. Then we build two types of query sets for evaluation under both zero-effort attack scenario and statistical attack scenario. The query set for zero-effort attack evaluation includes the other 130 participants' samples as illegitimate samples while the query set for statistical attack evaluation consists of 10,000 forged feature vectors as illegitimate samples. The experiment is repeated 10 times and the average performance is demonstrated to reduce variance that might stem from the randomness.

To better illustrate the impact of the statistical attack, we subtract the EER under zero-effort attack from EER under statistical attack for each participant, to highlight the changes in contrast to the zero-effort attack. EER change below zero means statistical attack does not have much influence on this participant, instead, it is even easier to distinguish than zero-effort attack samples. The greater EER change is, the more effective is the statistical attack for this user. We demonstrate the empirical distribution function of all users' EER changes in Figure 8. We can observe that handcrafted behavioral features have a different pattern with other types of features, e.g., the average EER change is +24.47%, in other words, if a user builds her model with 5% EER in the scenario of zero-effort attack, the user is likely to get an EER about 29% (=5+24.47) under the statistical attack. Compared with zero-effort attack, over 40% of participants have EER increase more than 20% under statistical attack (Figure 8 left). This result indicates that the statistical attack successfully ruins the behavioral features both in breadth and depth, which is aligned with the prior work [41, 42, 52].

However, for the deep features and physiological features, the negative impact of the statistical attack is much weaker (EER changes less than +3%). Curves in the figure show that over 60% of participants are not negatively affected. For those affected, the EER changes are limited to 30%. This result indicates value ranges of the same

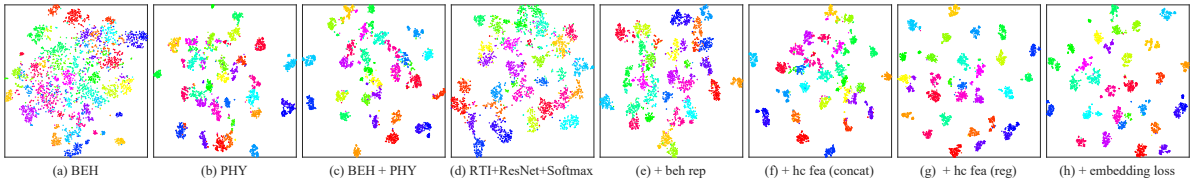


Fig. 9. The visualization of eight types of different models by t-SNE. (a)-(h) are explained in Table 4.

Table 4. Performance of different models is evaluated. Models in light gray are proposed in this paper. EERs in bold are significantly lower than the results of Model c ($p < 0.05$).

ID	Models	Average EERs (%)		
a.	Behavioral features (BEH)	12.2		
b.	Physiological features (PHY)	4.30		
c.	BEH + PHY features	4.16		
		STI	GTI	TDS
d.	RTI + ResNet + Softmax loss	5.71	5.71	5.71
e.	+ behavior representation	4.19	4.37	4.52
f.	+ hc features (concatenation)	3.22	3.26	3.43
g.	+ hc features (regularization)	**1.83**	**2.06**	**2.25**
h.	+ embedding loss (triplet + center)	**1.58**	**1.70**	**1.98**

feature dimension from different users are widely separated and different feature dimensions are not highly correlated. The deep features without regularization depict the strongest capability of defending against statistical attacks, especially the TDS representation scheme proposed in this paper being almost immunized from the attack (EER changes -3.52%). On the other hand, only a small proportion (about 20%) of participants become more vulnerable under statistical attack for regularized deep features. Even though the regularized ones perform slightly worse than TDS without regularization, they are almost as effective (EER changes less than +1.31%) as the physiological features (EER changes +0.58%) regarding defending the statistical attack, which may owe to the fact that the proposed architecture effectively decouples the features so that the statistical attack only results in small EER gains. In general, the proposed representations with or without regularization perform better than or equivalent to physiological features, which have been demonstrated most effective in [50], when prompted with statistical attacks.

7.5 Ablation Study

The proposed framework incorporates three key ideas, i.e., behavior representation, handcrafted feature regularization, and embedding loss, therefore, an ablation study is conducted to better understand the effect of each idea in the proposed deep learning framework.

Totally, eight types of models (*a*-*h*) are built and evaluated with the same samples under the unified training and inference protocol. The first three models (Model *a*, *b*, and *c*) employ the handcrafted behavioral or physiological or combined features to verify user. The 4th model (Model *d*) is the plain version of our framework, with no behavior representations (RTI as input), no handcrafted features for regularization ($f_o = f_d^t$ in Figure 3), and no embedding loss (only softmax loss). Then we sequentially augment Model d with the behavior representations (Model *e*), handcrafted feature regularization (Model *g*), and embedding loss (Model *h*). We also employ a commonly used fusion method (concatenation: $f_s = [f_d; f_h]$ in Figure 3, Model f) to compare with Model *g* regarding combining

with the handcrafted features [29, 30]. Model *e*, *g*, and *h* each brings in an important idea of our proposal while the other models are presented for comparison. 30 users outside the training set are selected and the diverse feature representations of their samples are qualitatively visualized using t-SNE [34] in Figure 9.

We summarize the results in Table 4. It shows the ranking of performances is basically $a<d<b\approx c\approx e<f<g<h$. This demonstrates the inclusion of each proposed idea leads to some performance improvements. Wilcoxon signed-rank tests for pairwise comparisons at the 0.05 alpha value are also employed to investigate the performance differences across models. Since Model *c* and *b* achieve equivalent EERs (which indicates a simple stack of handcrafted features can not reach their full capability) and Model *d* performs not as well as Model *c*, we set Model *c* as the baseline and compare it with Model *e-h*. As shown in the table, the EERs (in bold) of Model *g* and *h* are significantly lower than the baseline. Although Model *d* does not surpass the accuracy of Model *c*, it is notable that the behavior representations are proved effective in the contrast between Model *d* and *e*. In addition, as discussed in Section 7.1, Model *e* could also achieve better results than *b* if more irrelevant training samples are involved. As for the fusion with handcrafted features, both concatenation and FRN assist to decrease EERs. However, the comparison between *f* and *g* suggests that an appropriate approach for fusion could further enhance the effectiveness of handcrafted features by a large margin, such as integrating them into the network to supervise the deep features learning process (FRN). Model *h* as the full version of the proposed framework achieves the best performances, and we reckon the improvement should be attributed to the impact of adding embedding loss, which supervises the learning in a more fine-grained manner.

7.6 Comparison with State-of-the-Art

Experiments are conducted to compare with State-of-the-Art methods on our dataset in Table 5. Since the proposed FRN decouples the handcrafted feature regularization from the deep feature extraction, the deep feature extractor thus can be replaced by other deep models. Therefore, in this section, we aim to investigate 1) whether employing SOTA models in our framework improves the authentication performances and 2) whether FRN assists these SOTA models to reach a higher accuracy when combined with handcrafted features. More precisely, the deep models for comparison come from two categories: touch dynamic specific models (light-weight) for biometrics and general-purpose models (heavy-weight) for computer vision tasks. To make the comparisons fair, in the experiments, we use the unified training and inference protocol to build datasets for all models. Each model is evaluated under two conditions: using deep features alone vs. integrating handcrafted features through FRN. We discover that by leveraging SOTA deep models in our framework, better performances could be achieved; and FRN further boosts the accuracy of these augmented models and leads to lower EERs.

For touch dynamic specific models, we consider FCBBS and Sig2Vec as the deep feature extractors in our framework. As shown in Table 1, both models are proposed recently with state-of-the-art performances on touch behavioral authentication and are lightweight to be executed on mobile devices. **FCBBS** [48] is a dedicated 1D-CNN network to learn features from touch behaviors. It consists of 16 layers that employ 1D-convolutional, 1D-max pooling, dense, BatchNorm, and dropout layers as the basic modules, with the exponential linear unit (ELU) as the activation function. **Sig2Vec** [27] is built to extract holistic representations from time functions of dynamic sequences such as touch gestures and signatures. It consists of six convolutional layers, each followed by the scaled exponential linear unit (SELU), and three max-pooling layers. Two SP modules are also employed to fuse the outputs of varied convolutional layers. The official implementation of Sig2Vec[2] is used in our experiments. For FCBBS, we implement the network with PyTorch based on the details from the original paper. Parameters that are not mentioned in the paper are set with the standard values.

Besides the models designed for touch dynamic, we also explore using general-purpose models Swin Transformer and RepVGG in our framework. **Swin Transformer** [31] is one of state-of-the-art computer vision

[2]https://github.com/LaiSongxuan/SynSig2Vec

Table 5. Comparison with existing methods on our dataset. Models in light gray are proposed in this paper. EERs in bold are significantly lower than their deep feature counterparts ($p < 0.05$).

Extractors	Average EERs (%)	
	dp features alone	+ hc features reg
FCBBS	4.12	**2.37** (↓ 1.75)
Sig2Vec	3.91	**2.32** (↓ 1.59)
Swin-B	4.31	**1.55** (↓ 2.76)
RepVGG-B2	3.86	**1.63** (↓ 2.23)
ResNet-50	4.03	**1.70** (↓ 2.43)

models utilizing the hierarchical transformer design and shift window approach. Specifically, we use the base model Swin-B (about 4× the model size of ResNet-50) as the our deep feature extractor. Swin-B is built on four network stages with 2, 2, 18, 2 Swin Transformer blocks, respectively. A block consists of a shifted window based multi-head self-attention module, a 2-layer MLP with Gaussian error linear units (GELU) in between, LayerNorm layers, and residual connections. **RepVGG** [14] is a simple but powerful architecture of the convolutional neural network in image recognition and the version RepVGG-B2 (about 3× the model size of ResNet-50) is used as the extractor. RepVGG-B2 has five stages with 1, 4, 6, 16, 1 layers respectively. Each layer only employs convolution, residual connection, and batch normalization operations for the sake of efficiency. The official implementations of Swin[3] and RepVGG[4] are used in the experiments.

In Table 5, when only using deep features alone, RepVGG and Sig2Vec slightly outperform ResNet-50. The results show that by leveraging SOTA deep models in our framework, the overall performance could be further improved. More importantly, both light- and heavy-weight models benefit from the regularization of handcrafted features. The FRN still works for SOTA models and consistently yields significant improvements ($p < .05$) over those models without regularization, indicating the scalability and flexibility of FRN when applied to different deep models. Nevertheless, the EER drops of general computer vision models are much larger than the touch-specific models. It is probably because FCBBS and Sig2Vec are dedicatedly designed and optimized with the structure to emphasize the touch dynamic attributes, leaving some overlaps between the deep and handcrafted features. Yet Swin and RepVGG focus on learning from the images and hence extract deep features more distinct from the handcrafted ones. Interestingly, we observe the highest EER drop (2.76%) and the best performance (1.55%) comes from the Swin-B. We speculate that the poor performance of deep feature alone by Swin-B goes to the fact that such large model is hard to train sufficiently and is underfitting to some extent. However, the presence of handcrafted feature alleviates this issue and Swin-B thus reach a better level of its potential capability. This is aligned with our previous discussion that FRN assists the network to converge more easily.

8 DISCUSSION

8.1 Other Advanced Attacks

In the previous section, we examined the authentication performance under the statistical attack which utilized the statistics of the population to generate forged samples. For biometric techniques, replay attacks and adversarial attacks are two other types of advanced attacks that greatly threaten the security of techniques such as fingerprint and face recognition. In a replay attack, an attacker records and replays the genuine user's previously successful

[3]https://github.com/microsoft/Swin-Transformer
[4]https://github.com/DingXiaoH/RepVGG

action to fool the system. In an adversarial attack, the attacker attempts to feed the model specially crafted input to fail the algorithm.

More specifically, in our proposed architecture, replay attack and adversarial attack can be conducted via touchscreen by performing the physical actions, or inside the system by injecting deceptive samples. For the former approach, the replay attack requires attackers to duplicate a fake hand with the same hand geometry as the victim's, and move it to reproduce the genuine gestures. The adversarial attack requires physically adding deliberate modifications over the swiping process. Both tasks are extremely hard to complete if not impossible for two reasons. First, reproducing the complete hand movement of the genuine sample in a 3D environment is difficult either by video record or by feature vector. Besides, even the hand movements are reconstructed, robots cannot exactly replicate the complicated motor skills as expected due to constrained movements. As for the latter injection approach, it requires to some extent to breach the operation system, which is out of the scope of our authentication method.

8.2 Computational Latency

It should be noted that deep models with more parameters cost longer time for computation, so it is essential to consider the computational latency with smartphone hardware. Since we employ a general model which could be trained and loaded before the genuine user registers, the latency mainly includes two parts at the inference stage: FRN for regularized feature extraction and distance-based KNN classifier for decision making.

For feature extraction, by leveraging efficient programming frameworks which utilize advanced model compression and architecture-aware optimization techniques, the computational latency of deep model can be largely reduced [36]. For example, with the 2.45GHz CPU Snapdragon 835 and 6GB shared memory, a ResNet-50 network can finish the inference of a single input within 230 ms on the smartphone. Using a 710MHz GPU Adreno 540 could further speed up the computation to 60 ms. The latency of the KNN classifier for decision-making is also short (less than 200 ms) due to low-dimension feature vectors and tiny search space. Moreover, replacing ResNet-50 with lightweight models such as MobileNets [22] or student network out of knowledge distillation [21] could further reduce the time cost by a large step. Therefore, the computational latency of our method is not a troublesome issue for smartphone users.

8.3 Cross-device Authentication

The dataset we use only includes touch samples collected with a single mobile device, therefore, we investigate the generalization capability of the proposed method across different devices in the future. For example, in the world of IoT devices, if the templates are registered with a smartphone touchscreen, how well does the model work on the screen of a touchpad in the vehicle? Different touchscreens have varied hardware with their own resolution, sensitivity, and sampling rate, and current methods relying on the measure of pixels may not be reliable enough across devices. It is critical to design and implement a device-irrelevant model across multiple devices, saving the cost of repeatedly registering on different devices. It could leverage the physical measurements in the real world as features rather than the digital pixels, or automatically adjust the values of pixels according to characteristics of the touchscreen. Also, the network should be trained with the ability to eliminate the differences caused by varied hardware.

9 CONCLUSION

Although the behavioral authentication methods secure smartphones in a more privacy-preserving manner, yet the behavioral uncertainty in the wild affects the reliability of these techniques and hinders the deployments. To this end, we propose a deep feature regularization framework that integrates the handcrafted with the deep features to enhance the security and stability of behavioral authentication schemes. We exploit to incorporate

handcrafted features in the two phases of the deep learning process. We first present three behavior representation methods for explicitly encoding semantic handcrafted features into the raw touch data. Then we propose a novel deep Feature Regularization Net (FRN) to fuse the complementary information in handcrafted and deep features. The proposed framework leverages the invariant information in semantic handcrafted features to supervise the deep feature learning process, and then selectively combines the feature sets via a feature re-weighting mechanism based on the input. Finally, a mixed loss is formulated for fine-grained supervision.

Extensive experiments demonstrate that FRN outperforms the previous approaches of handcrafted or deep features even with fewer samples for training and registration. With appropriate feature extractors, such as SOTA models in the domain of computer vision, the performance of our proposal can be further improved. More importantly, FRN works for diverse deep models and further boosts the accuracy by a large margin. The results also show that the proposed method is better than or equivalent to the existing methods in facing challenges posed by behavioral variability and statistical attacks, and suggest a promising approach to improve the effectiveness and usability of behavioral authentication methods for mobile devices.

ACKNOWLEDGMENTS

We are grateful to Professor Xiaohong Guan for his kind support of this work and anonymous reviewers for their insightful comments. This work was funded in part by the National Key R&D Program of China under Grant No. 2018AAA0101501 and the Science and Technology project of SGCC (State Grid Corporation of China): fundamental theory of human-in-the-loop hybrid-augmented Intelligence for power grid dispatch and control.

REFERENCES

[1] Mohammed Abuhamad, Ahmed Abusnaina, DaeHun Nyang, and David Mohaisen. 2020. Sensor-based Continuous Authentication of Smartphones' Users Using Behavioral Biometrics: A Contemporary Survey. *IEEE Internet of Things Journal* 8, 1 (2020), 65–84.

[2] Alejandro Acien, John V Monaco, Aythami Morales, Ruben Vera-Rodriguez, and Julian Fierrez. 2020. Typenet: Scaling up keystroke biometrics. *arXiv preprint arXiv:2004.03627* (2020).

[3] Margit Antal and László Zsolt Szabó. 2016. Biometric authentication based on touchscreen swipe patterns. *Procedia Technology* 22 (2016), 862–869.

[4] Adam J Aviv, Katherine L Gibson, Evan Mossop, Matt Blaze, and Jonathan M Smith. 2010. Smudge attacks on smartphone touch screens. *Woot* 10 (2010), 1–7.

[5] Philip Bontrager, Aditi Roy, Julian Togelius, Nasir Memon, and Arun Ross. 2018. Deepmasterprints: Generating masterprints for dictionary attacks via latent variable evolution. In *2018 IEEE 9th International Conference on Biometrics Theory, Applications and Systems (BTAS)*. IEEE, 1–9.

[6] Léon Bottou. 2012. Stochastic gradient descent tricks. In *Neural networks: Tricks of the trade*. Springer, 421–436.

[7] Zhongmin Cai, Chao Shen, Miao Wang, Yunpeng Song, and Jialin Wang. 2013. Mobile authentication through touch-behavior features. In *Chinese Conference on Biometric Recognition*. Springer, 386–393.

[8] Changhao Chen, Stefano Rosa, Yishu Miao, Chris Xiaoxuan Lu, Wei Wu, Andrew Markham, and Niki Trigoni. 2019. Selective sensor fusion for neural visual-inertial odometry. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10542–10551.

[9] Yimin Chen, Jingchao Sun, Rui Zhang, and Yanchao Zhang. 2015. Your song your way: Rhythm-based two-factor authentication for multi-touch mobile devices. In *2015 IEEE conference on computer communications (INFOCOM)*. IEEE, 2686–2694.

[10] Zhenghua Chen, Chaoyang Jiang, Shili Xiang, Jie Ding, Min Wu, and Xiaoli Li. 2019. Smartphone sensor-based human activity recognition using feature fusion and maximum full a posteriori. *IEEE Transactions on Instrumentation and Measurement* 69, 7 (2019), 3992–4001.

[11] Penny Chong, Yuval Elovici, and Alexander Binder. 2019. User authentication based on mouse dynamics using deep neural networks: A comprehensive study. *IEEE Transactions on Information Forensics and Security* 15 (2019), 1086–1101.

[12] Alexander De Luca, Alina Hang, Frederik Brudy, Christian Lindner, and Heinrich Hussmann. 2012. Touch me once and i know it's you! implicit authentication based on touch screen patterns. In *proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 987–996.

[13] Google Developers. [n.d.]. Android API reference. https://developer.android.com/reference

[14] Xiaohan Ding, Xiangyu Zhang, Ningning Ma, Jungong Han, Guiguang Ding, and Jian Sun. 2021. Repvgg: Making vgg-style convnets great again. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 13733–13742.
[15] Alaa El Masri, Harry Wechsler, Peter Likarish, Christopher Grayson, Calton Pu, Dalal Al-Arayed, and Brent ByungHoon Kang. 2015. Active authentication using scrolling behaviors. In *2015 6th International Conference on Information and Communication Systems (ICICS)*. IEEE, 257–262.
[16] Julian Fierrez, Ada Pozo, Marcos Martinez-Diaz, Javier Galbally, and Aythami Morales. 2018. Benchmarking touchscreen biometrics for mobile authentication. *IEEE transactions on information forensics and security* 13, 11 (2018), 2720–2733.
[17] Mario Frank, Ralf Biedert, Eugene Ma, Ivan Martinovic, and Dawn Song. 2012. Touchalytics: On the applicability of touchscreen input as a behavioral biometric for continuous authentication. *IEEE transactions on information forensics and security* 8, 1 (2012), 136–148.
[18] Matteo Gadaleta and Michele Rossi. 2018. Idnet: Smartphone-based gait recognition with convolutional neural networks. *Pattern Recognition* 74 (2018), 25–37.
[19] Alex Graves and Jürgen Schmidhuber. 2005. Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural networks* 18, 5-6 (2005), 602–610.
[20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
[21] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015).
[22] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).
[23] Andrej Karpathy, Justin Johnson, and Li Fei-Fei. 2015. Visualizing and understanding recurrent networks. *arXiv preprint arXiv:1506.02078* (2015).
[24] Hassan Khan, Urs Hengartner, and Daniel Vogel. 2018. Evaluating attack and defense strategies for smartphone pin shoulder surfing. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–10.
[25] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
[26] Pavel Korshunov and Sébastien Marcel. 2018. Deepfakes: a new threat to face recognition? assessment and detection. *arXiv preprint arXiv:1812.08685* (2018).
[27] Songxuan Lai, Lianwen Jin, Yecheng Zhu, Zhe Li, and Luojun Lin. 2021. SynSig2Vec: Forgery-free Learning of Dynamic Signature Representations by Sigma Lognormal-based Synthesis. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2021).
[28] Gaël Le Lan and Vincent Frey. 2019. Securing smartphone handwritten pin codes with recurrent neural networks. In *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2612–2616.
[29] Yantao Li, Peng Tao, Shaojiang Deng, and Gang Zhou. 2021. DeFFusion: CNN-based Continuous Authentication Using Deep Feature Fusion. *ACM Transactions on Sensor Networks (TOSN)* 18, 2 (2021), 1–20.
[30] Chenhao Lin and Ajay Kumar. 2018. A CNN-based framework for comparison of contactless to contact-based fingerprints. *IEEE Transactions on Information Forensics and Security* 14, 3 (2018), 662–676.
[31] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. *arXiv preprint arXiv:2103.14030* (2021).
[32] Hao Luo, Youzhi Gu, Xingyu Liao, Shenqi Lai, and Wei Jiang. 2019. Bag of tricks and a strong baseline for deep person re-identification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*. 0–0.
[33] Hao Luo, Wei Jiang, Youzhi Gu, Fuxu Liu, Xingyu Liao, Shenqi Lai, and Jianyang Gu. 2019. A strong baseline and batch normalization neck for deep person re-identification. *IEEE Transactions on Multimedia* 22, 10 (2019), 2597–2609.
[34] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, Nov (2008), 2579–2605.
[35] Natalia Neverova, Christian Wolf, Griffin Lacey, Lex Fridman, Deepak Chandra, Brandon Barbello, and Graham Taylor. 2016. Learning human identity from motion patterns. *IEEE Access* 4 (2016), 1810–1820.
[36] Wei Niu, Xiaolong Ma, Yanzhi Wang, and Bin Ren. 2019. 26ms inference time for resnet-50: Towards real-time execution of all dnns on smartphone. *arXiv preprint arXiv:1905.00571* (2019).
[37] Shih Yin Ooi and Andrew Beng-Jin Teoh. 2019. Touch-stroke dynamics authentication using temporal regression forest. *IEEE Signal Processing Letters* 26, 7 (2019), 1001–1005.
[38] Youcef Ouadjer, Mourad Adnane, and Nesrine Bouadjenek. 2021. Feature Importance Evaluation of Smartphone Touch Gestures for Biometric Authentication. In *2020 2nd International Workshop on Human-Centric Smart Environments for Health and Well-being (IHSH)*. IEEE, 103–107.
[39] Napa Sae-Bae, Kowsar Ahmed, Katherine Isbister, and Nasir Memon. 2012. Biometric-rich gestures: a novel approach to authentication on multi-touch devices. In *proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 977–986.
[40] Napa Sae-Bae, Nasir Memon, Katherine Isbister, and Kowsar Ahmed. 2014. Multitouch gesture-based authentication. *IEEE transactions on information forensics and security* 9, 4 (2014), 568–582.

[41] Abdul Serwadda and Vir V Phoha. 2013. Examining a large keystroke biometrics dataset for statistical-attack openings. *ACM Transactions on Information and System Security (TISSEC)* 16, 2 (2013), 1–30.
[42] Abdul Serwadda and Vir V Phoha. 2013. When kids' toys breach mobile phone security. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 599–610.
[43] Muhammad Shahzad, Alex X Liu, and Arjmand Samuel. 2013. Secure unlocking of mobile touch screen devices by simple gestures: you can see it but you can not do it. In *Proceedings of the 19th annual international conference on Mobile computing & networking*. 39–50.
[44] Richard Shay, Saranga Komanduri, Adam L Durity, Phillip Huh, Michelle L Mazurek, Sean M Segreti, Blase Ur, Lujo Bauer, Nicolas Christin, and Lorrie Faith Cranor. 2016. Designing password policies for strength and usability. *ACM Transactions on Information and System Security (TISSEC)* 18, 4 (2016), 1–34.
[45] Chao Shen, Yuanxun Li, Yufei Chen, Xiaohong Guan, and Roy A Maxion. 2017. Performance analysis of multi-motion sensor behavior for active smartphone authentication. *IEEE Transactions on Information Forensics and Security* 13, 1 (2017), 48–62.
[46] Chao Shen, Yong Zhang, Xiaohong Guan, and Roy A Maxion. 2015. Performance analysis of touch-interaction behavior for active smartphone authentication. *IEEE Transactions on Information Forensics and Security* 11, 3 (2015), 498–513.
[47] Michael Sherman, Gradeigh Clark, Yulong Yang, Shridatt Sugrim, Arttu Modig, Janne Lindqvist, Antti Oulasvirta, and Teemu Roos. 2014. User-generated free-form gestures for authentication: Security and memorability. In *Proceedings of the 12th annual international conference on Mobile systems, applications, and services*. 176–189.
[48] Dai Shi, Dan Tao, Jiangtao Wang, Muyan Yao, Zhibo Wang, Houjin Chen, and Sumi Helal. 2021. Fine-Grained and Context-Aware Behavioral Biometrics for Pattern Lock on Smartphones. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 5, 1 (2021), 1–30.
[49] Zdeňka Sitová, Jaroslav Šeděnka, Qing Yang, Ge Peng, Gang Zhou, Paolo Gasti, and Kiran S Balagani. 2015. HMOG: New behavioral biometric features for continuous authentication of smartphone users. *IEEE Transactions on Information Forensics and Security* 11, 5 (2015), 877–892.
[50] Yunpeng Song, Zhongmin Cai, and Zhi-Li Zhang. 2017. Multi-touch authentication using hand geometry and behavioral information. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 357–372.
[51] Yunpeng Song, Cori Faklaris, Zhongmin Cai, Jason I Hong, and Laura Dabbish. 2019. Normal and Easy: Account Sharing Practices in the Workplace. *Proceedings of the ACM on Human-Computer Interaction* 3, CSCW (2019), 1–25.
[52] Valeriu-Daniel Stanciu, Riccardo Spolaor, Mauro Conti, and Cristiano Giuffrida. 2016. On the effectiveness of sensor-enhanced keystroke dynamics against statistical attacks. In *proceedings of the sixth ACM conference on data and application security and privacy*. 105–112.
[53] Ruben Tolosana, Ruben Vera-Rodriguez, Julian Fierrez, and Javier Ortega-Garcia. 2020. BioTouchPass2: Touchscreen Password Biometrics Using Time-Aligned Recurrent Neural Networks. *IEEE Transactions on Information Forensics and Security* 15 (2020), 2616–2628.
[54] TSYS. 2018. U.S. Consumer Payment Study. (2018). https://www.tsys.com/Assets/TSYS/downloads/rs_2018-us-consumer-payment-study.pdf
[55] Verizon. 2020. 2020 Data Breach Investigations Report. https://enterprise.verizon.com/resources/reports/dbir/
[56] Jiayun Wang, Patrick Virtue, and Stella X Yu. 2017. Successive Embedding and Classification Loss for Aerial Image Classification. *arXiv preprint arXiv:1712.01511* (2017).
[57] Jian Wang, Feng Zhou, Shilei Wen, Xiao Liu, and Yuanqing Lin. 2017. Deep metric learning with angular loss. In *Proceedings of the IEEE International Conference on Computer Vision*. 2593–2601.
[58] Yandong Wen, Kaipeng Zhang, Zhifeng Li, and Yu Qiao. 2016. A discriminative feature learning approach for deep face recognition. In *European conference on computer vision*. Springer, 499–515.
[59] Shangxuan Wu, Ying-Cong Chen, Xiang Li, An-Cong Wu, Jin-Jie You, and Wei-Shi Zheng. 2016. An enhanced deep feature representation for person re-identification. In *2016 IEEE winter conference on applications of computer vision (WACV)*. IEEE, 1–8.
[60] Hui Xu, Yangfan Zhou, and Michael R Lyu. 2014. Towards continuous and passive authentication via touch biometrics: An experimental study on smartphones. In *10th Symposium On Usable Privacy and Security ({SOUPS} 2014)*. 187–198.
[61] Xiangyu Xu, Jiadi Yu, Yingying Chen, Qin Hua, Yanmin Zhu, Yi-Chao Chen, and Minglu Li. 2020. TouchPass: towards behavior-irrelevant on-touch user authentication on smartphones leveraging vibrations. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. 1–13.
[62] Daksha Yadav, Naman Kohli, Akshay Agarwal, Mayank Vatsa, Richa Singh, and Afzel Noore. 2018. Fusion of handcrafted and deep learning features for large-scale multiple iris presentation attack detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*. 572–579.
[63] Muhammad Rehman Zafar and Munam Ali Shah. 2016. Fingerprint authentication and security risks in smart devices. In *2016 22nd International Conference on Automation and Computing (ICAC)*. IEEE, 548–553.
[64] Ahmad Zairi Zaidi, Chun Yong Chong, Zhe Jin, Rajendran Parthiban, and Ali Safaa Sadiq. 2021. Touch-based continuous mobile device authentication: State-of-the-art, challenges and opportunities. *Journal of Network and Computer Applications* (2021), 103162.
[65] Heng Zhang, Vishal M Patel, Mohammed Fathy, and Rama Chellappa. 2015. Touch gesture-based active user authentication using dictionaries. In *2015 IEEE Winter Conference on Applications of Computer Vision*. IEEE, 207–214.

[66] Xinchen Zhang, Yafeng Yin, Lei Xie, Hao Zhang, Zefan Ge, and Sanglu Lu. 2020. TouchID: User Authentication on Mobile Devices via Inertial-Touch Gesture Analysis. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 4, 4 (2020), 1–29.

[67] Xi Zhao, Tao Feng, and Weidong Shi. 2013. Continuous mobile authentication using a novel graphic touch gesture feature. In *2013 IEEE sixth international conference on biometrics: theory, applications and systems (BTAS)*. IEEE, 1–6.